

大数据雨课堂习题

• 第一章

• 一、判断题

- 物联网的发展目标是“物物相连”。（对）
- 大数据、云计算和物联网代表了 IT 领域最新的技术发展趋势，三者相辅相成，既有联系又有区别。大数据根植于云计算，云计算提供了海量数据的存储和管理能力；大数据是物联网的重要来源，物联网数据分析为大数据技术提供支撑。（错）
- 错误原因：物联网是大数据的重要来源，云计算为大数据提供技术基础
- 数据和信息是相同的概念。（错）
- 错误原因：数据是对客观事物的符号记录，信息是经加工后有价值的数据，二者概念不同。

• 二、多选题

- 大数据技术的不同层面包括（ABCD）
 - A 数据采集与预处理
 - B 数据存储与管理
 - C 数据处理与分析
 - D 数据安全和隐私保护
- 大数据的影响体现在（ABC）
 - A 科学研究方式的变化：数据密集型科学
 - B 思维方式上的颠覆：全样而非抽样，效率而非精确，相关而非因果
 - C 大数据决策方式成为一种新的决策方式
 - D 在信息软件学科中深入研究形成大数据学科的全部知识体系
- 大数据的 4 个 V 包含（ABCD）
 - A 数据量大
 - B 数据种类繁多
 - C 处理速度快
 - D 价值密度低
 - E 价值密度高

• 三、单选题

- 结构化数据是指（B）
 - A 文本文档
 - B 存储在关系数据库中的数据
 - C 音频视频

- D 位置信息
- 下列哪个存储单位最大 (A)
 - A ZB
 - B GB
 - C EB
 - D PB
 - $PB < EB < ZB < YB$

• 第二章

• 一、单选题

- 关于配置命令，以下选项中描述错误的是 (A)
 - **A Start - dfs 可以启动 hdfs 和 yarn**
 - B Stop - dfs 可以停止 hdfs 服务
 - C Hdfs 配置好启动前应格式化名称节点
 - D Yarn 相关的进程是 ResourceManager 和 NodeManager
- 关于 Hadoop 伪分布式安装，以下选项中描述错误的是 (B)
 - A 副本数应配置为 1
 - **B start - all.sh 不可以启动 hdfs 和 yarn**
 - C Jps 用来查看已启动的进程，完全启动的 hadoop 应该包含 5 个 hadoop 进程
 - D 伪分布式安装是在一台机器上模拟一个小的集群。
- 关于 Hadoop 的安装，以下选项中描述错误的是 (D)
 - A 分布式需要配置 SSH 免密服务
 - B 需要 JDK 的支持
 - C 可以配置为单机模式
 - **D 单机模式就是伪分布式模式**
- 关于 Zookeeper，以下选项中描述错误的是 (B)
 - A 高效可靠的协同工作系统
 - **B 在 Hadoop 和关系数据库之间交换数据**
 - C 提供分布式锁服务
 - D 用于构建分布式应用，减轻分布式应用程序所承担的协调任务
- 不可以提供流计算功能是 (B)
 - A Spark
 - **B MapReduce**
 - C Storm
 - D Flink
- 下来没有自定义类 SQL 语言的工具是 (B)

- A Pig
- **B MapReduce**
- C Hive
- D Spark
- 以下选项中，关于 Hadoop 特性的描述错误的是 (D)
 - A 采用冗余数据存储方式
 - B 可以高效的处理 PB 级数据
 - C 对失败的任务可以自动进行重新分配
 - **D 只能运行在 Linux 系统中**
- HBase 是 (C)
 - A 关系数据库
 - B 键值数据库
 - **C 分布式列式数据库**
 - D 文档数据库
- HDFS 是 (A)
 - **A 分布式文件系统**
 - B 分布式并行计算框架
 - C 批处理框架
 - D 资源管理器
- Hadoop1.0 不包括下列哪个组件 (C)
 - A HDFS
 - B MapReduce
 - **C Yarn**

• 第三章

• 一、选择题

- 单选题 HDFS 核心的概念是 (C)
 - A. 单元
 - B. 节点
 - **C. 块**
 - D. 文件
- 单选题 下面哪个程序负责 HDFS 数据文件存储。(C)
 - A. NameNode
 - B. Jobtracker
 - **C. DataNode**
 - D. secondaryNameNode

- **单选题** HDFS 和其它分布式文件系统的最大区别就是 (D)
 - A. 可以调整冗余数据的位置
 - B. HDFS 在部署时都提供了客户端
 - C. HDFS 是一个部署在集群上的分布式文件系统
 - **D. HDFS 集群中只有一个命名空间**
- **单选题** 名称结点在分布式文件系统中又叫做 (D)
 - A. 数据节点
 - B. 从节点
 - C. 集群节点
 - **D. 主节点**
- **单选题** 如何解决名称节点运行期间 EditLog 不断变大导致的名称节点启动缓慢的问题? (A)
 - **A. 第二名称节点**
 - B. FsImage
 - C. 数据节点协议
 - D. RPC(Remote Procedure Call)
- **单选题** Hadoop 平台中, 创建一个或多个指定文件夹的命令是 (C)。
 - A. `hadoop fs -ls`
 - B. `hadoop fs -chown`
 - **C. `hadoop fs -mkdir`**
 - D. `hadoop fs -copyFromLocal`
- **单选题** Hadoop 平台中, 显示指定文件夹详细信息的命令是 (A)
 - **A. `hadoop fs -ls`**
 - B. `hadoop fs -chown`
 - C. `hadoop fs -mkdir`
 - D. `hadoop fs -copyFromLocal`
- **单选题** 当客户端读取数据时, 从名称节点获得数据块不同副本的存放位置列表, 列表中包含了副本所在的数据节点, 客户端通过什么判断读取那个副本? (C)
 - A. 块编号
 - B. 地理位置
 - **C. 机架 ID**
 - D. 名称节点指定
- **单选题** 名称节点命名空间不足的问题通过下列哪种技术可以解决? (A)
 - **A. HDFS Federation**
 - B. HDFS HA

- C. SecondaryNameNode
- 单选题 副本复制的方式是 (B)
 - A. 客户端同时在多个数据节点上执行写操作
 - B. 数据节点之间流水线复制
 - C. 名称节点将数据分别发送到多个数据节点
 - D. 客户端直接将数据发送给名称节点
- 多选题 下列各项中，属于名称节点的功能的有 (ABC)
 - A. 管理分布式文件系统系统的命名空间
 - B. 记录分布式的文件系统系统中的每个文件中各块的位置信息
 - C. 管理数据块 (Block) 映射信息
 - D. 在运行过程中合并 FSImage 和 Editlog 的内容 //第二名称节点的任务
- 多选题 下列各项中，属于 DataNode 的功能的是 (ABC)
 - A. 存储实际的数据块
 - B. 执行数据块的读 / 写操作
 - C. 周期性向 NameNode 汇报心跳信息和数据块信息
 - D. 自动配置副本策略
- 多选题 下列各项中，属于 SecondaryNameNode 的功能的有 (AD)
 - A. 辅助恢复 NameNode
 - B. NameNode 出现故障的时候直接接管系统
 - C. 解决单点故障
 - D. 定期合并 fsimage 和 edits, 并推送给 NameNode
- 多选题 下列属于正确的副本存放策略的是 (ABCD)
 - A. 第一个副本：放置在上传文件的数据节点；如果是集群外提交，则随机挑选一台磁盘不太满、CPU 不太忙的节点
 - B. 第二个副本：放置在与第一个副本不同的机架的节点上
 - C. 第三个副本：与第一个副本相同机架的其他节点上
 - D. 更多副本：随机节点
- 多选题 下列对于 HDFS 错误与恢复的机制中，描述正确的有 (ABCD)
 - A. 名称节点出错时，就可以根据备份服务器 SecondaryNameNode 中的 FSImage 和 Editlog 数据进行恢复。
 - B. 当数据节点发生故障或者网络发生断网时，名称节点就无法收到来自一些数据节点的心跳信息，这时这些数据节点就会被标记为“不可用”节点，上面的所有数据都会被标记为不可读，名称节点不会再给它们发送任何 I/O 请求
 - C. 由于一些数据节点的不可用，会导致一些数据块的副本数量的不冗余，因此，名称节点会定期检查这种情况，一旦发现某个数据块的副本数量小于冗余因子，就会启动数据冗余复制，为它生成新的副本

- D. 当客户端读取文件的时候，会先读取该信息文件，然后利用该信息文件对每个读取的数据块进行校验，如果校验出错，客户端就会请求到另外一个数据节点读取该块数据，并且向名称节点报告这个文件块有错误，名称节点会定期检查并且重新复制这个块

• 二、主观题

- 主观题 请阐述 HDFS 在不发生故障的情况下读文件的过程。

- 打开文件：客户端通过FileSystem的open()方法打开文件，获得输入流FSDataInputStream。
 - 获取数据块信息：通过ClientProtocol.getBlockLocations()远程调用名称节点，获得文件开始部分数据块的位置，名称节点返回保存该数据块的所有数据节点的地址并排序。
 - 读取数据：输入流根据排序结果选择距离客户端最近的数据节点建立连接并读取数据，数据从数据节点读到客户端，数据块读取完毕时关闭连接
 - 查找下一个数据块：再次调用ClientProtocol.getBlockLocations()查找下一个数据块，重复读取过程，直到文件读取完毕

```
try {
    Configuration conf = new Configuration();
    conf.set("fs.defaultFS", "hdfs://localhost:9000");
    conf.set("fs.hdfs.impl", "org.apache.hadoop.hdfs.DistributedFileSystem");    FileSystem fs =
    FileSystem.get(conf);
    Path file = new Path("test");
    FSDataInputStream getIt = fs.open(file);
    BufferedReader d = new BufferedReader(new InputStreamReader(getIt));
    String content = d.readLine(); //读取文件一行        System.out.println(content);

    d.close(); //关闭文件
    fs.close(); //关闭hdfs
} catch (Exception e) {
    e.printStackTrace();
}
```

- 主观题 请阐述 HDFS 在不发生故障的情况下写文件的过程。

- 创建文件：RPC远程调用名称节点，在文件系统的命名空间中新建一个文件，名称节点会执行文件是否存在、客户端权限等检查
 - 数据分包：数据被分成一个个分包，放入FSDataOutputStream对象的内部队列
 - 申请数据节点：FSDataOutputStream向名称节点申请保存数据块的若干数据节点，这些数据节点形成一个数据流管道 - 数据传输：队列中的分包最后被打包成数据包，发往数据流管道中的第一个数据节点，第一个数据节点将数据包发送到第二个节点，依此类推，形成“流水线复制”
 - 确认与关闭：接收到数据的数据节点向发送者发送“确认包”，确认包逆流而上到达客户端，客户端收到应答时将对应的分包从内部队列移除。最后，FSDataOutputStream调用ClientProtocol.complete()方法通知名称节点关闭文件

```

try {
    Configuration conf = new Configuration();
    conf.set("fs.defaultFS", "hdfs://localhost:9000");
    conf.set("fs.hdfs.impl", "org.apache.hadoop.hdfs.DistributedFileSystem");    FileSystem fs =
    FileSystem.get(conf);
    byte[] buff = "Hello world".getBytes(); // 要写入的内容
    String filename = "test"; // 要写入的文件名    FSDDataOutputStream os = fs.create(new
    Path(filename));    os.write(buff, 0, buff.length);
    System.out.println("Create:" + filename);
    os.close();
    fs.close();
} catch (Exception e) {
    e.printStackTrace();
}

```

- **主观题** 请阐述第二名称节点辅助名称节点进行镜像文件合并的过程。
 - EditLog 与 FsImage 的合并操作。每隔一段时间，第二名称节点会和名称节点通信，请求其停止使用 EditLog 文件，暂时将新到达的写操作添加到一个新的文件 EditLog.new 中。
 - 然后，第二名称节点把名称节点中的 FsImage 文件和 EditLog 文件拉回本地，再加载到内存中；对二者执行合并操作，使 FsImage 保持最新。
 - 合并结束后，第二名称节点会把合并后得到的最新的 FsImage.ckpt 文件发送到名称节点。
 - 名称节点收到后，会用最新的 FsImage.ckpt 文件去替换旧的 FsImage 文件，同时用 EditLog.new 文件去替换 EditLog 文件，从而减小了 EditLog 文件的大小。

• 第四章

• 一、选择题

- 单选题
 - A. Spark
 - B. MapReduce
 - C. **BigTable**
 - D. Hive
- HBase 是针对 (C) 的开源实现。
- 单选题
 - A. Spark
 - B. **MapReduce**
 - C. BigTable
 - D. Hive
- HBase 用来处理 HBase 中海量数据的是 (B)
- 单选题
 - A. Pig
 - B. Hive
 - C. Tez

- D. ZooKeeper
- HBase 用来实现稳定服务和失败回复，作为协同服务的是 (D)
- 单选题
 - A. HDFS
 - B. MapReduce
 - C. BigTable
 - D. Hive
- HBase 用来实现高可靠的底层存储的是 (A)
- 单选题
 - A. 键值
 - B. 关系
 - C. 列族
 - D. 文档
- HBase 是下列哪种类型的数据库? (C)
- 单选题
 - A. 行键
 - B. 列族
 - C. 列限定符
 - D. 单元格
- HBase 中表的行是根据谁的值的字典序进行维护的? (A)。
- 单选题
 - A. Region 名
 - B. RegionID
 - C. 表名
 - D. 开始主键
- HBase 中每个 Region 都有一个 (B) 来标识他的唯一性。
- 单选题
 - A. MemStore
 - B. EditLog
 - C. StoreFile
 - D. HLog
- Region 服务器中配备了一个 (D) 文件，所有更新的数据都需要先计入此文件才能写入缓存。
- 单选题
 - A. MemStore

- B. EditLog
- C. StoreFile
- D. HLog
- 用户读取数据时，需要先访问 (A)，找不到数据时，才去磁盘上寻找。
- 单选题
 - A. Region 是根据表中行的主键进行的分区
 - B. HBase 查找用户 Region 时需要先去 HMaster 查找 Root 表地址
 - C. Region 拆分操作非常快，接近瞬间，因为拆分之后的 Region 读取的仍然是原存储文件。
 - D. Zookeeper 为 HBase 集群中各进程提供分布式协调服务。各 RegionServer 将自己的信息注册到 Zookeeper 中，HMaster 据此感知各个 RegionServer 的健康状态。
- 下列说法不正确的是 (B)
- 多选题
 - A. ZooKeeper
 - B. -ROOT - 表
 - C. .META 表
 - D. HMaster
- 下列各项中，属于三级寻址需要访问到的有 (ABC)
- 多选题
 - A. 表名
 - B. 开始主键
 - C. RegionID
 - D. 时间戳
- 下列各项中，组成 Region 标识符的是 (ABC)
- 多选题
 - A. 行键
 - B. 列族
 - C. 列限定符
 - D. 时间戳
- 下列各项中，HBase 用来进行单元格定位需要的有 (ABCD)
- 多选题
 - A. Region 中有多个 Store，每个 Store 对应一个列族的存储
 - B. 每个 Store 中，只有一个 MemStore，作为内存中的缓存，保存最近更新的数据
 - C. 每个 Store 中有多个 StoreFile 文件
 - D. MemStore 中的数据会周期性的刷新到磁盘中形成 StoreFile。

- 下列属于正确的是 (ABCD)
- 多选题
 - A. 每个 Region 服务器只有一个 HLog 文件，所有 Region 对象共用一个 HLog。
 - B. StoreFile 文件过多是需要合并，文件过大时需要分裂。
 - C. HMaster 负责表和 Region 的管理工作，负载很低。
 - D. RegionServer 负责维护分配给自己的 Region，并响应客户端读写请求。
- 下列说法正确的是 (ABCD)

• 第七章

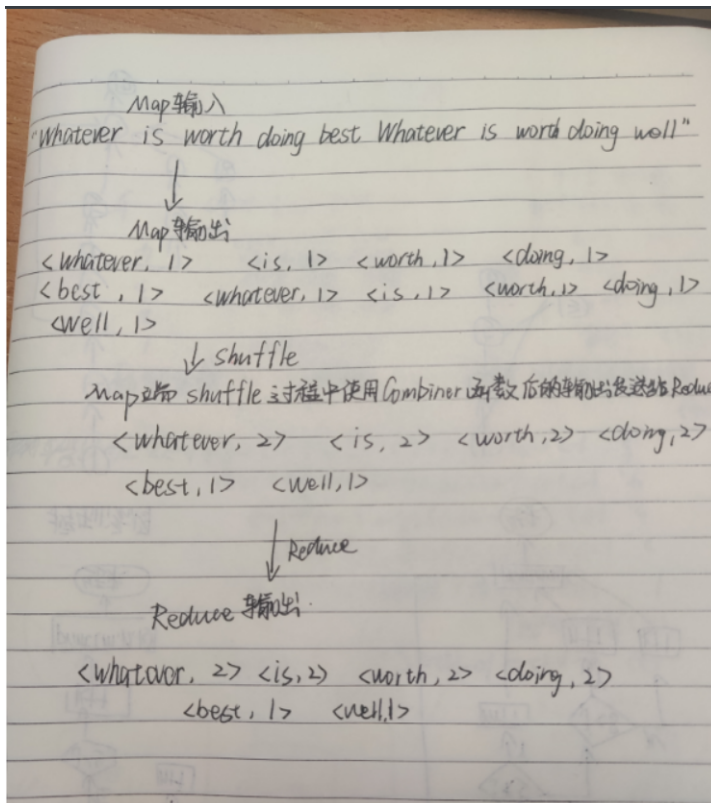
• 一、选择题

- 单选题
 - A. 数据向计算靠拢
 - B. 自行进行工作调度
 - C. 计算向数据靠拢
 - D. 无需负载均衡
- 属于 MapReduce 的设计理念的是 (C)
- 单选题
 - A. Key
 - B. Text
 - C. 键值对
 - D. 文件块
- Map 和 Reduce 函数都是以 (C) 作为输入。
- 单选题
 - A. 分而治之
 - B. 流计算
 - C. 分布式存储
 - D. 批处理
- MapReduce 的核心思想是 (A)
- 单选题
 - A. 一个 Map 函数只能转换出一个键值对
 - B. Map 转换出的键没有唯一性
 - C. 同一输入元素，可以通过一个 Map 任务生成具有相同键的多个键值对
 - D. 键不可以作为输出的身份标识
- Map 函数将输入的元素转换成键值对，下列说法不正确的是 (A)
- 单选题
 - A. 通过 Hash 函数对数据进行排序

- B. 对数据举行分区
- C. 将输入的一系列具有相同 Key 的键值对以某种方式组合起来，输出处理后的键值对
- D. 对数据进行解析得到键值对
- Reduce 函数的任务是 (C)
- 单选题
 - A. 分区
 - B. 排序
 - C. 合并归并
 - D. 切分
- Shuffle 过程不包括下列哪个过程？ (D)
- 单选题
 - A. Task
 - B. Partitioner
 - C. Job
 - D. Combiner
- 每个 MapReduce 程序都需要一个 (C)
- 单选题
 - A. 内存
 - B. 磁盘
 - C. ResourceManager
 - D. Container
- MapReduce 处理海量数据的时候，中间数据存储在 (B)
- 多选题
 - A. 编写 Map 处理逻辑
 - B. 编写 Reduce 处理逻辑
 - C. 编写 main 方法
 - D. 提前生成输出文件夹
- 编写 MapReduce 程序时， (ABC) 。

• 二、主观题

- 主观题 试画出使用 MapReduce 对英语段落 “Whatever is worth doing best Whatever is worth doing well” 进行单词统计过程



- 主观题 分析描述 Shuffle 的执行过程。
 - 1.Map端的Shuffle过程 Map输出结果首先被写入缓存，缓存满时启动溢写操作，将缓存的数据进行分区，然后将每个分区进行排序和合并，之后写入磁盘文件。Map任务全部结束之前，这些溢写文件会归并成一个大的磁盘文件，然后通知相应的Reduce任务来领取属于自己处理的数据
 - 2.Reduce端的Shuffle过程 Reduce任务从Map端的不同Map机器领会属于自己处理的数据，对数据进行归并后交给Reduce处理。
- 主观题 使用 MapReduce 完成对下表中及格同学的筛选，描述程序设计实现过程（文字描述即可，不用写代码）
 - 输入数据与拆分假设原始数据是每行一条记录，格式为学号,姓名,成绩（如001,丁一,90），MapReduce 先将输入文件拆分为若干输入分片（Input Split），每个分片分配给一个 Map 任务处理。
 - Mapper 阶段：筛选与输出
 - 处理逻辑：读取一行输入，按分隔符拆分出学号、姓名、成绩字段；将成绩字段转为整数，判断是否 ≥ 60 ；若及格，通过 context.write() 输出键值对（Key 可设为任意标记，如空字符串；Value 为完整记录，如001,丁一,90）。
 - Shuffle 阶段（框架自动处理）MapReduce 会自动完成 Shuffle 流程：
 - 分区（Partition）：根据 Key 决定数据发往哪个 Reduce；
排序（Sort）：对 Map 输出的 Key-Value 按 Key 排序；
合并
 - Reducer 阶段：直接输出结果
 - 处理逻辑：Reduce 任务接收 Shuffle 后的数据，遍历所有 Value（即及格同学记录）；通过 context.write() 直接输出每条记录

- 输出结果最终输出文件中，仅保留成绩 ≥ 60 分的同学记录（如 001,丁一,90、002,白二,60），完成及格同学筛选需求。

• 第八章

• 一、主观题

- 请详细说明HDFS2.0相对1.0的改进都有哪些？
- 1.设计了HDFS HA，提供名称节点热备机制，避免了1.0时单一名称节点，存在单点失效的问题
- 2.设计了HDFS Federation，管理多个命名空间，避免了1.0单一命名空间，无法实现资源隔离的问题
- YARN由哪些组件构成，他们的功能分别是什么？
- **ResourceManager**
- (1).处理客户端请求 (2).启动/监控ApplicationMaster
- (3).监控NodeManager (4).资源分配与调度
- **ApplicationMaster**
- (1).为应用程序申请资源，并分配给内部任务
- (2).任务调度、监控与容错
- **NodeManager**
- (1).单个节点上的资源管理
- (2).处理来自ResourceManger的命令 (3).处理来自ApplicationMaster的命令
- **ResourceManager(RM)**是一个全局的资源管理器，负责整个系统的资源管理和分配，主要包括两个组件，即调度器(Scheduer)和应用程序管理器(ApplicationsManager)。容器(Container)作为动态资源分配单位
- YARN工作的具体流程是什么？
- 步骤1:用户编写客户端应用程序，向YARN提交应用程序，提交的内容包括ApplicationMaster程序、/启动ApplicationMaster的命令、用户程序等。
- 步骤2:YARN中的ResourceManager负责接收和处理来自客户端的请求，为应用程序分配一个容器在该容器中启动一个ApplicationMaster。
- 步骤3:ApplicationMaster被创建后会首先向ResourceManager注册
- 步骤4:ApplicationMaster采用轮询的方式向ResourceManager申请资源
- 步骤5:ResourceManager以“容器”的形式向提出申请的ApplicationMaster分配资源。
- 步骤6:在容器中启动任务(运行环境、脚本)
- 步骤7:各个任务向ApplicationMaster汇报自己的状态和进度
- 步骤8:应用程序运行完成后，ApplicationMaster向ResourceManager的应用程序管理器注销并关闭自己。

• 第九章

• 一、判断题

- **判断题** Hive 是构建于 Hadoop 顶层的数据仓库工具，本身不存储和处理数据（√）
- **判断题** Hive 需把 HiveQL 语句转换成 MapReduce 任务运行（√）
- **判断题** Hive 同时支持导入单条数据和批量数据（×）
- **错误原因：** Hive 主要用于批量数据处理，对单条数据导入支持不佳，通常适用于大规模数据的批量操作场景。
- **判断题** 传统数据库有分区功能，Hive 不支持分区功能，用索引加快查询（×）
- **错误原因：** Hive 支持分区功能，合理分区可提升查询效率，且 Hive 一般不用索引（索引在 Hive 中效果有限，常用分区、分桶优化）。
- **判断题** Hive 的元数据存储模块 Metastore 是独立关系型数据库（√）

• 二、主观题

- **主观题** 数据湖与数据仓库的区别有哪些？
 - **数据存储：** 数据湖存原始数据，查询分析时加工；数据仓库存经加工、符合预定义架构的数据。
 - **处理方式：** 数据湖用 ELT（提取、加载、转换）；数据仓库用 ETL（提取、转换、加载）。
 - **灵活性：** 数据湖无需预定义架构，灵活性高；数据仓库需预定义架构，灵活性低。
目标用户： 数据湖面向数据科学家、分析师、开发人员；数据仓库面向业务分析师、决策者。
 - **分析目的：** 数据湖用于探索性分析、机器学习等；数据仓库用于标准报告、业务智能分析等。
 - **数据治理与质量：** 数据湖治理相对宽松，依赖用户；数据仓库治理严格，保障质量一致。
 - **数据模式：** 数据湖是模式在读（Schema - on - Read）；数据仓库是模式在写（Schema - on - Write）。
 - **可扩展性：** 数据湖易适应数据量增长，扩展性高；数据仓库受架构设计限制，扩展性弱。
 - **成本：** 数据湖初期存原始数据成本低；数据仓库因预处理和架构维护，成本高。
 - **查询性能：** 数据湖查询时加工数据，复杂分析适用但耗时；数据仓库数据经优化，查询快速

特征	数据湖	数据仓库
数据类型	结构化、半结构化、非结构化	主要是结构化
数据存储	原始数据存储，等待被查询和分析时加工	经过加工的数据，符合预先定义的架构
处理方式	ELT(提取、加载、转换)	ETL(提取、转换、加载)
灵活性	高，可以存储任何形式的数据，不需要预先定义架构	较低，需要预先定义数据架构
目标用户	数据科学家、分析师、开发人员	业务分析师、决策者
分析目的	探索性分析、机器学习、大数据处理	标准报告、业务智能、绩效指标分析
数据治理与质量	相对较松，依赖用户对数据的掌握程度	较为严格，确保数据质量和一致性
数据模式	模式在读 (Schema-on-Read)	模式在写 (Schema-on-Write)
可扩展性	高，容易适应数据量的增长	受到架构和设计的限制
成本	通常成本较低，尤其在初期存储大量原始数据时	高，由于需要大量预处理和维护预定义的架构
查询性能	可能需要更多的处理时间，因为数据在查询时才进行加工，但适合复杂分析	快速，数据已经过优化以支持快速查询

● 主观题 数据仓库的概念特性是什么？

- 核心功能：数据仓库聚焦分析、报告与大数据场景。
- 数据来源：数据仓库整合多个来源，经抽取和标准化汇聚。
- Schema 特点：数据仓库采用非标准化 Schema，适配多样数据分析需求。
- 写入方式：数据仓库按批处理计划批量写入。
- 存储设计：数据仓库用列式存储，利于高速查询、低开销访问。
- 读取优化：数据仓库为减 I/O、提吞吐做优化，契合大数据分析读取模式。

● 主观题 详细说明 Hive 的系统架构组成及各部分包含内容？

- Hive主要由以下3个模块组成，:用户接口模块、驱动模块以及元数据存储模块。
 - 1、用户接口模块
 - 用户接口模块包括CL(命令行客户端工具)、Hive网页接口(HiveWebInterface, HWI)JDBC、ODBC、Thrift Server等，用来实现 外部应用对Hive的访问。
 - 2、驱动模块(Driver)
 - 驱动模块(Driver)包括解析器，编译器、优化器、执行器等，驱动模块负责把 HiveQL语句转换成一系列MapReduce作业
 - 3、元数据存储模块(Metastore)
 - 元数据存储模块(Metastore)是一个独立的关系数据库。Hive的元数据存储的关系数据库中

● 第十章

● 一、判断题

- 判断题 Spark 的计算模式属于 MapReduce (√)
- 判断题 Spark 提供内存计算，中间结果放内存，迭代运算效率高 (√)
- 判断题Spark 新生态系统只包含 Spark Core、Spark SQL 等组件 (×)
- 错误原因：Spark 生态系统除这些组件，还可与其他工具集成，且组件也在发展扩展，并非“只包含”。

- **判断题** Spark 只能部署在 YARN 上提供大数据解决方案 (×)
- 错误原因: Spark 可部署在多种资源管理器上, 如 Standalone、Mesos 等, 不局限于 YARN。
- **判断题** Spark 基于 DAG 的任务调度执行机制优于 Hadoop MapReduce 迭代执行机制 (√)
- **判断题** Spark 中高度优先共享内存模型是 RDD, 依赖关系由 DAG 反映 (√)

• 二、主观题

- **主观题** 写出下列Spark中的概念: (1)RDD (2)DAG (3)Executor (4)Application (5)Task (6)Job (7)Stage (8)宽依赖 (9)窄依赖
 - RDD: 分布式内存的一个抽象概念, 提供一种高度受限的共享内存模型
 - 本质是可并行处理的只读数据集 (分布式数据集)
 - RDD只能基于稳定的物理存储中的数据创建或在其他RDD上执行确定的转换操作创建
 - DAG (有向无环图): 反映RDD之间的依赖关系
 - Executor: 工作进程, 负责在Spark作业Job中运行任务Task
 - 应用Application: 用户编写的Spark应用程序
 - 作业Job: 一个作业Job包含多个RDD及作用于相应RDD上的各种操作
 - 阶段Stage: 作业Job的基本调度单位
 - 一个作业Job分为多组任务Task, 每组任务称为阶段或任务集
 - 任务Task: 运行在Executor上的工作单元
 - 宽依赖: 子 RDD 的一个分区依赖于父 RDD 的多个分区 (如 groupByKey、reduceByKey)。
 - 窄依赖: 子 RDD 的每个分区最多依赖于父 RDD 的一个分区 (如 map、filter)。
- **主观题** RDD 操作的分类有哪些?
 - action 和 transformation
 - action用于执行计算并指定输出的形式
 - transformation用于指定RDD之间的依赖关系
- **主观题** Spark 运行架构包括哪些?
 - 集群管理器 (Cluster Manager)
 - 运行作业任务的工作节点 (Worker Node)
 - 每个应用的任务控制节点 (Driver)
 - 每个工作节点上负责具体任务的执行进程 (Executor)
 - 资源管理器
- **主观题** Spark 运行基本流程是什么?
 - 1 为应用构建起基本的运行环境, 即由Driver创建一个SparkContext, 进行资源的申请、任务的分配和监控
 - 2 资源管理器为Executor分配资源, 并启动Executor进程。

- 3 SparkContext根据RDD的依赖关系构建DAG图，DAG图提交给DAGScheduler解析成Stage，然后把一个个TaskSet提交给底层调度器TaskScheduler处理；Executor向SparkContext申请Task，TaskScheduler将Task发放给Executor运行，并提供应用程序代码
- 4 Task在Executor上运行，把执行结果反馈给TaskScheduler，然后反馈给DAGScheduler，运行完毕后写入数据并释放所有资源

• 第十一章

- **判断题**在 Web 应用、网络监控、传感监测等领域，兴起了一种新的数据密集型应用 —— 静态数据，即数据以大量、快速、时变的流形式持续到达。（×）
- 错误原因：这种数据形式是流数据，不是静态数据。
- **判断题**流数据适合采用批量计算，因为流数据适合用传统的关系模型建模。（×）
- 错误原因：流数据适合流计算，传统关系模型不适合流数据实时、连续等特点。
- **判断题**流计算：实时获取来自不同数据源的海量数据，经过实时分析处理，获得有价值的信息。（√）
- **判断题**流计算秉承一个基本理念，即数据的价值随着时间的流逝而增加。（×）
- 错误原因：流计算理念是数据价值随时间流逝而降低，需及时处理。
- **判断题**Storm 是免费、开源的分布式实时计算系统，可简单、高效、可靠地处理大量的流数据（√）
- **判断题**流计算的处理流程一般包含三个阶段：数据实时采集、数据实时计算、实时查询服务（√）
- **判断题**数据实时采集阶段通常采集多个数据源的海量数据，需要保证实时性、低延迟与稳定可靠（√）
- **判断题**Spark Streaming 的基本原理是将实时输入数据流以时间片（秒级）为单位进行拆分，然后经 Spark 引擎以类似批处理的方式处理每个时间片数据（√）
- **判断题**Spark Streaming 和 Storm 最大的区别在于，Spark Streaming 可以实现毫秒级的流计算，而 Storm 无法实现毫秒级响应（×）
- 错误原因：Storm 可实现毫秒级流计算，Spark Streaming 是秒级，该表述颠倒。
- **判断题**流计算可应用多个场景中，如实时业务分析，流计算带来的实时性特点，可以大大增加实时数据的价值，为业务分析带来质的提升（√）

• 第十二章

- **判断题**Flink 的主要特性包括：批流一体化、精密的状态管理、事件时间支持以及精确一次的状态一致性保障等。（√）
- **判断题**Flink 不仅可以运行在包括 YARN、Mesos、Kubernetes 等在内的多种资源管理框架上，还支持在裸机集群上独立部署。在启用高可用选项的情况下，它不存在单点失效问题。（√）
- **判断题**为了高效地实现流处理架构，一般需要设置消息传输层、流处理层和批处理层。（×）

- 错误原因：批处理层不需要
- **判断题** Flink 支持高吞吐、高延迟、高性能。 (×)
- 错误原因：Flink 是低延迟，不是高延迟。
- **判断题** Flink 支持流处理和批处理，具有良好的容错性，具有独立的内存管理。 (√)
- **判断题** 流处理架构的核心是使各种应用程序互连在一起的消息队列，消息队列连接应用程序，并作为新的共享数据源，这些消息队列取代了从前的大型集中式数据库。 (√)
- **判断题** 流处理架构需要具备低延迟、高吞吐和高性能的特性，而目前从市场上已有的产品来看，不只有 Flink 可以满足要求。 (×)
- 错误原因：只有flink，其他都fw

• 第十三章

- **判断题** 许多非图结构的大数据，也常常会被转换为图模型后进行分析。 (√)
- **判断题** 关联性计算是大数据计算的核心 —— 通过获得数据的关联性，可以从噪音很多的海量数据中抽取有用的信息。 (√)
- **判断题** 一次 BSP (Bulk Synchronous Parallel Computing Model, 又称“大同步”模型) 计算过程包括一次全局超步 (所谓的超步就是计算中的一次迭代)。 (×)
- 错误原因：一次 BSP 计算过程包含多个全局超步。
- **判断题** 每个超步主要包括四个组件：局部计算、通讯、路障和栅栏同步。 (×)
- 错误原因：每个超步主要包括局部计算、通讯、栅栏同步，无“路障”。
- **判断题** Pregel 是一种基于 BSP 模型实现的并行图处理系统，主要用于图遍历、最短路径、PageRank 计算等等。 (√)
- **判断题** Pregel 图计算模型以有向图为输入，计算过程由一系列超步迭代而成。 (√)